MCA Semester – IV

Project - Interim Report

Name	Deepak Jain
USN	241VMTR01377
Elective	Cloud Computing
Date of Submission	15/07/2026

A study on "EchoMateLite: Design and Deployment of a Cloud-Based Lightweight Social Media Platform on Amazon Web Services (AWS)"

A Project submitted to Jain Online (Deemed-to-be University)

In partial fulfillment of the requirements for the award of:

Master of Computer Application

Submitted by:
Deepak Jain

USN: 241VMTR01377

Under the guidance of:

Madhulika Praveen Dubey
(Faculty-JAIN Online)

Jain Online (Deemed-to-be University)
Bangalore
2025–26

DECLARATION

I, *Deepak Jain,* hereby declare that the Interim Project Report titled *"EchoMateLite: Design and Deployment of a Cloud-Based Lightweight Social Media Platform on Amazon Web Services (AWS)"* has been prepared by me under the guidance of *Madhulika Praveen Dubey.* I declare that this Project work is towards the partial fulfillment of the University Regulations for the award of the degree of Master of Computer Application by Jain University, Bengaluru. I have undergone a project for a period of Eight Weeks. I further declare that this Project is based on the original study undertaken by me and has not been submitted for the award of any degree/diploma from any other University / Institution.

Place: Gurugram _____Deepak Jain_____

Date: 15/07/2026 *USN: 241VMTR01377*

EXECUTIVE SUMMARY

This Interim Report documents the progress, architectural decisions, and current implementation status of "EchoMateLite," a lightweight, cloud-based social media platform being developed as part of the MCA Semester IV Capstone Project in Cloud Computing at Jain Online (Deemed-to-be University). The project aims to design, develop, and deploy a fully functional social media web application on Amazon Web Services (AWS), demonstrating practical proficiency in full-stack MERN development and cloud infrastructure management.

EchoMateLite is built using the MERN stack – MongoDB for the database layer, Express.js as the backend framework, React.js for the frontend user interface, and Node.js as the server runtime. This technology combination was selected for its unified JavaScript ecosystem, high performance in handling concurrent requests, and strong alignment with modern web development industry standards. The application is designed to support core social media functionalities: secure user registration and authentication, user profile management with picture uploads, text-based post creation, and a chronological news feed.

At this interim stage, the project has successfully completed the backend foundation and the user authentication module. A production-ready Express.js server has been established following the Model-View-Controller (MVC) architectural pattern. The MongoDB Atlas database cluster has been provisioned on AWS infrastructure in the Mumbai (ap-south-1) region as a free M0 sandbox, and a successful connection has been verified. Complete user registration and login APIs have been implemented with JSON Web Token (JWT) based authentication, along with a reusable authentication middleware for protecting private routes.

The project source code is version-controlled using Git and hosted on GitHub. All implemented features have been tested using Postman, confirming successful API responses including HTTP 201 (Created) for user registration and proper JWT token generation. The cloud architecture has been designed to leverage the AWS Free Tier, with the backend planned for Amazon EC2 (t2.micro), the React frontend for Amazon S3 with

CloudFront distribution, and HTTPS enforcement through an Application Load Balancer with AWS Certificate Manager.

The remaining phases of the project include implementing the Post data model and associated APIs, building the React.js frontend application, and deploying the complete application stack on AWS. This report provides a comprehensive overview of the work completed thus far, the rationale behind the chosen technology stack and AWS services, and a detailed solution architecture with cost analysis.

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

CHAPTER 1
INTRODUCTION

INTRODUCTION

1.1 Background Information of the Project

Social media platforms have fundamentally transformed the way individuals communicate, share information, and build communities in the digital age. Platforms such as Facebook, Twitter, and Instagram serve billions of users worldwide, facilitating real-time interaction through profiles, posts, feeds, and messaging systems. However, these commercial platforms are built on massive, proprietary infrastructure stacks involving thousands of microservices, complex distributed systems, and hardware investments running into billions of dollars. This level of complexity makes them unsuitable as learning references or templates for academic cloud computing projects.

There exists a clear academic and practical need for a lightweight social media platform that demonstrates the core principles of modern web application development and cloud deployment without the overhead of commercial-scale complexity. Such a platform would serve as an ideal capstone project for a Master of Computer Application (MCA) program, combining full-stack web development with hands-on cloud infrastructure management.

"EchoMateLite" is designed to address this need. It is a lightweight, cloud-deployable social media platform that implements the essential functionalities of a social network – user authentication, profile management, post creation, and a public news feed – while remaining simple enough to be developed, tested, and deployed by a single developer within the constraints of a semester-long capstone project. The platform is built using the MERN stack (MongoDB, Express.js, React.js, Node.js) and is architected for deployment on Amazon Web Services (AWS), leveraging the AWS Free Tier to demonstrate cost-efficient cloud architecture.

This project is undertaken as part of the MCA Semester IV Capstone Project in Cloud Computing at Jain Online (Deemed-to-be University), Bangalore. The project provides practical, end-to-end exposure to the complete software development lifecycle: from requirement analysis and architectural design, through implementation and testing, to cloud deployment and documentation.

1.2 Goals and Objectives

The overarching goal of EchoMateLite is to design, develop, and successfully deploy a lightweight cloud-based social media platform on Amazon Web Services (AWS), demonstrating practical proficiency in full-stack MERN development, AWS cloud architecture design, and end-to-end cloud deployment.

The specific objectives of the project are:

1.	To design and develop a fully functional MERN-stack web application (EchoMateLite) that supports user registration, login, profile management, post creation, and a dynamic news feed.
2.	To design a scalable, cost-efficient AWS cloud architecture capable of hosting the EchoMateLite application with high availability and low latency.
3.	To integrate relevant AWS services – including Amazon EC2, Amazon S3, MongoDB Atlas, Amazon CloudFront, and Elastic Load Balancing – to form a complete, production-grade cloud solution.
4.	To implement fundamental security practices including HTTPS enforcement via AWS Certificate Manager, secure password hashing using bcryptjs, JWT-based authentication, and IAM-based access control following the principle of least privilege.
5.	To evaluate the performance, cost, and scalability of the deployed AWS architecture and document the findings comprehensively with architecture diagrams and cost analysis.

Table 1.1: Project Technology Stack Summary

Layer	Technology	Purpose
Frontend	React.js	Single Page Application (SPA) for user interface
Backend	Node.js + Express.js	RESTful API server
Database	MongoDB Atlas	NoSQL cloud database (AWS Mumbai region)
Authentication	JWT + bcryptjs	Token-based auth with password hashing

1.3 Key Requirements of the Project

To meet its stated objectives, EchoMateLite must fulfill a set of essential features and functionalities. These requirements are derived directly from the university-assigned problem statement. The following table summarizes the key functional requirements:

Table 1.2: Key Functional Requirements

Requirement	Description	Priority
User Registration	Users must be able to create new accounts by providing their name, email address, and a password. The system must validate all inputs and reject duplicate email addresses.	High
User Login	Registered users must be able to log in using their email and password. The system must verify credentials against securely hashed passwords stored in the database.	High
JWT Authentication	Upon successful login or registration, the system must issue a JSON Web Token (JWT) that the client uses to authenticate subsequent API requests. A middleware layer must protect all private routes.	High
User Profile	Authenticated users must be able to view and edit their profile information, including their name, biography, and profile picture.	High
Post Creation	Authenticated users must be able to create text-based posts that are stored in the database and displayed in the news feed.	High
News Feed	The application must display a chronological list of all posts from all users, accessible to authenticated users.	Medium
Profile Picture Upload	Users must be able to upload a profile picture which is stored in Amazon S3 and served via a public URL.	Medium
HTTPS Enforcement	All traffic between the frontend and the backend must be encrypted using HTTPS, enforced via AWS Certificate Manager.	High

CHAPTER 2
LITERATURE REVIEW

LITERATURE REVIEW

2.1 Significance and Rationale for the Chosen Technology Stack

The technology stack for EchoMateLite was selected after careful evaluation of the project requirements, the university guidelines, and industry best practices. The MERN stack – MongoDB, Express.js, React.js, and Node.js – was chosen as the foundational technology due to several compelling advantages that directly align with the objectives of this capstone project.

Unified JavaScript Ecosystem:

The most significant advantage of the MERN stack is the use of JavaScript as the sole programming language across the entire application – from the frontend (React.js) to the backend (Node.js/Express.js) to the database query language (MongoDB). This eliminates the cognitive overhead of context-switching between different programming languages, accelerates development velocity, and reduces the learning curve. For a semester-constrained capstone project developed by a single student, this is a critical practical advantage.

Component-Based Frontend with React.js:

React.js was selected for the frontend layer because of its component-based architecture, which promotes code reusability and maintainability. React's Virtual DOM efficiently handles frequent UI updates – a critical capability for a social media application where the news feed must render and re-render dynamically as new posts arrive.

Event-Driven Backend with Node.js and Express.js:

Node.js provides a lightweight, event-driven, non-blocking I/O runtime that is highly efficient for handling numerous concurrent connections. Express.js, built on top of Node.js, provides a minimal and flexible web application framework that simplifies the creation of RESTful APIs through its robust routing system and middleware architecture.

Flexible Data Layer with MongoDB:

MongoDB, a document-oriented NoSQL database, was chosen for its schema flexibility and natural compatibility with JavaScript objects. MongoDB Atlas was selected

specifically because it can be hosted on AWS infrastructure in the Mumbai (ap-south-1) region, co-locating the database with the application server to minimize network latency.

Table 2.1: MERN Stack Component Analysis

Component	Technology	Role in EchoMateLite	Key Advantage
M – Database	MongoDB Atlas	Stores user profiles, posts, and session data as JSON documents	Schema flexibility, native JS compatibility
E – Backend Framework	Express.js	Handles HTTP requests, defines API routes, manages middleware pipeline	Minimal, fast, extensive middleware ecosystem
R – Frontend Library	React.js	Renders the user interface as a Single Page Application (SPA)	Virtual DOM, component reusability, large community
N – Runtime	Node.js	Executes JavaScript on the server, handles async I/O operations	Non-blocking I/O, event-driven, high concurrency

2.2 Significance and Rationale for the Chosen AWS Services

Amazon Web Services (AWS) was selected as the cloud platform for deploying EchoMateLite based on the university capstone project guidelines. Beyond this requirement, AWS offers the most comprehensive set of cloud services globally, an extensive Free Tier program suitable for academic projects, and deep integration capabilities between its services.

Amazon EC2 (Elastic Compute Cloud):

Amazon EC2 provides resizable compute capacity in the cloud through virtual server instances. For EchoMateLite, a t2.micro instance (1 vCPU, 1 GB RAM) running Ubuntu Server will host the Node.js/Express.js backend API. The t2.micro instance provides 750 hours of compute time per month at zero cost during the AWS Free Tier first 12 months.

Amazon S3 (Simple Storage Service):

Amazon S3 provides highly durable (99.999999999%) and available object storage. In EchoMateLite, S3 serves a dual purpose: (a) hosting the production build of the React.js frontend as a static website, and (b) storing user-uploaded profile pictures. The Free Tier provides 5 GB of standard storage.

Amazon CloudFront:
Amazon CloudFront is a Content Delivery Network (CDN) that caches and delivers content from edge locations worldwide, placed in front of the S3-hosted React frontend to reduce latency and enforce HTTPS using SSL/TLS certificates provisioned via AWS Certificate Manager.

MongoDB Atlas (Hosted on AWS):
MongoDB Atlas provides a fully managed MongoDB service natively compatible with the Mongoose ODM library. The M0 Sandbox tier (free) offers 512 MB of storage hosted on AWS infrastructure in the Mumbai (ap-south-1) region.

Elastic Load Balancer (ELB) + AWS Certificate Manager (ACM):
An Application Load Balancer (ALB) will be configured to handle SSL/TLS termination using a free certificate provisioned through AWS Certificate Manager (ACM), enforcing HTTPS for all API requests between the frontend and the backend.

AWS IAM (Identity and Access Management):
AWS IAM will be used to define roles and policies that govern access to all AWS resources. Following the principle of least privilege, each component will be granted only the permissions strictly necessary for its function.

Table 2.2: AWS Services Selected for EchoMateLite

AWS Service	Purpose in EchoMateLite	Free Tier Allowance
Amazon EC2	Host the Node.js/Express.js backend API server	750 hours/month (t2.micro)
Amazon S3	Host React frontend build + store profile pictures	5 GB storage, 20,000 GET requests
Amazon CloudFront	CDN for frontend delivery + HTTPS enforcement	1 TB data transfer out/month
MongoDB Atlas	Cloud-hosted NoSQL database for user and post data	512 MB storage (M0 Sandbox)
ALB + ACM	HTTPS termination and SSL/TLS certificate management	750 hours/month (new accounts)
AWS IAM	Role-based access control following least privilege	Always Free

2.3 A Brief Overview of the Problems Addressed by Using the Selected Stack

The combined selection of the MERN stack and AWS cloud services directly addresses several significant challenges encountered in modern web application development and deployment:

Problem 1: Development Complexity and Context Switching

Traditional web development stacks require developers to work with multiple programming languages across the frontend, backend, and database. The MERN stack eliminates this fragmentation by using JavaScript throughout the entire application, significantly reducing development time and minimizing bugs caused by context-switching.

Problem 2: Dynamic UI Performance

Social media applications require frequent and efficient UI updates as users scroll through feeds, receive notifications, and interact with posts. React.js solves this through its Virtual DOM mechanism, computing the minimal set of DOM changes needed, resulting in a fluid, responsive user interface.

Problem 3: Capital-Intensive Infrastructure

Hosting a web application on physical servers requires significant upfront capital expenditure. AWS eliminates this barrier entirely through its pay-as-you-go pricing model and Free Tier program. For EchoMateLite, the entire deployment architecture falls within the AWS Free Tier, reducing infrastructure cost to practically zero.

Problem 4: Authentication Security

Building a custom authentication system from scratch is notoriously error-prone. EchoMateLite implements industry-standard security practices: passwords hashed using bcryptjs with a cost factor of 10, user sessions managed through cryptographically signed JWTs, and authentication middleware validating every request to protected routes.

Problem 5: Scalability and Global Availability

A self-hosted application is limited by the capacity of a single server. The AWS architecture for EchoMateLite addresses this through Amazon CloudFront (serving the frontend from 400+ edge locations worldwide), MongoDB Atlas (supporting horizontal scaling through sharding), and the Application Load Balancer (able to distribute traffic across multiple EC2 instances as demand grows).

CHAPTER 3
SOLUTION ARCHITECTURE

SOLUTION ARCHITECTURE

3.1 Architecture Diagram

The architecture of EchoMateLite follows a decoupled, three-tier client-server model deployed across multiple AWS services. The architecture separates the presentation layer (React.js frontend), the business logic layer (Node.js/Express.js backend API), and the data layer (MongoDB Atlas) into independently deployable components connected through well-defined APIs and network configurations.

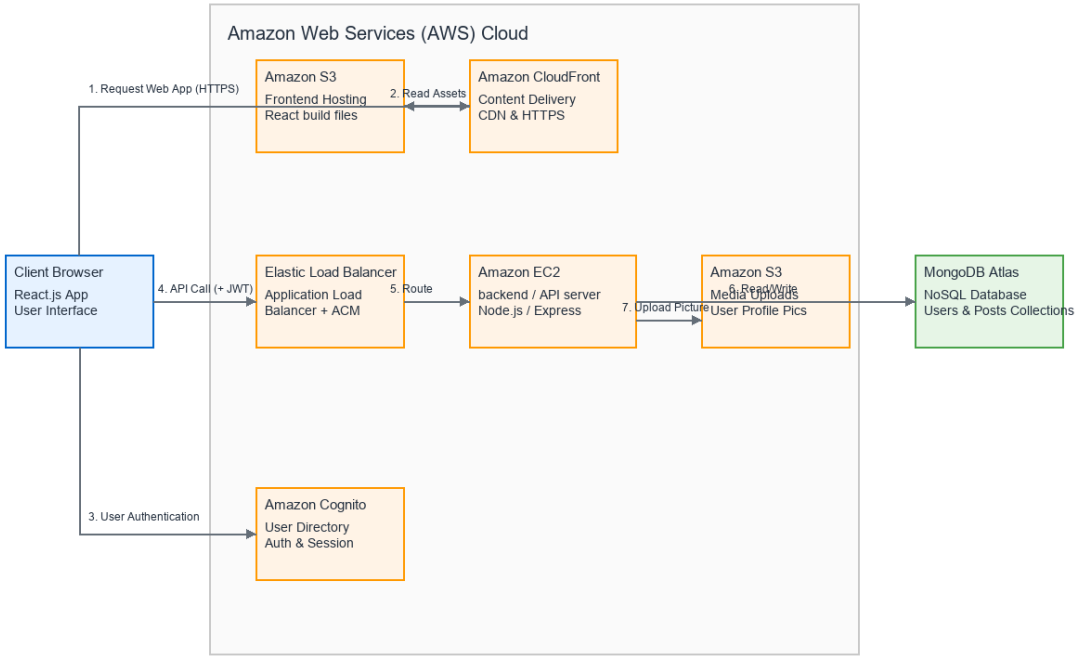

Figure 3.1: AWS Solution Architecture Diagram for EchoMateLite

Architecture Components and Data Flow:

Tier 1 – Presentation Layer (Frontend):

The user accesses EchoMateLite through a web browser. The React.js frontend is built into static HTML, CSS, and JavaScript files and hosted in an Amazon S3 bucket. Amazon CloudFront is placed in front of S3 as a CDN, caching the static assets at edge locations globally and enforcing HTTPS using an SSL/TLS certificate from AWS Certificate Manager (ACM).

Tier 2 – Business Logic Layer (Backend API):

The React frontend communicates with the Node.js/Express.js backend API through RESTful HTTP requests, with each request from an authenticated user including a JWT token in the Authorization header. The backend is hosted on an Amazon EC2 instance (t2.micro, Ubuntu Server) in the ap-south-1 (Mumbai) region, fronted by an Application Load Balancer for SSL/TLS termination.

Tier 3 – Data Layer (Database and Storage):
The data layer consists of: (a) MongoDB Atlas, a managed NoSQL database cluster hosted on AWS in the Mumbai region (M0 Sandbox, free tier), storing all application data including user accounts, hashed passwords, profiles, and posts; and (b) Amazon S3, which stores binary objects such as user-uploaded profile pictures.

Backend MVC Architecture (Implemented):
The backend follows the Model-View-Controller (MVC) architectural pattern with clear separation of concerns. The implemented folder structure is as follows:

Directory	Purpose	Current Contents
src/config/	Database and environment configuration	db.js – MongoDB Atlas connection
src/models/	Mongoose data schemas (the "Model" layer)	User.js – User schema with bcrypt hooks
src/controllers/	Business logic (the "Controller" layer)	authController.js – Register and Login
src/routes/	API endpoint definitions (URL mapping)	authRoutes.js – POST /register, /login
src/middleware/	Request interceptors	authMiddleware.js – JWT verification
src/utils/	Shared utility functions	generateToken.js – JWT generation

3.2 Cost Analysis and Financial Implications of the Selected Services
A primary design constraint of this capstone project is to minimize financial expenditure while utilizing enterprise-grade cloud services. The architecture has been intentionally

designed to leverage the AWS Free Tier, which provides new AWS accounts with 12 months of free access to a generous allocation of compute, storage, and networking resources.

Table 3.1: AWS Free Tier Cost Analysis

AWS Service	Free Tier Allocation	EchoMateLite Usage (Estimated)	Monthly Cost
Amazon EC2 (t2.micro)	750 hours/month	~720 hours (1 instance, always on)	$0.00
Amazon S3	5 GB storage, 20,000 GET, 2,000 PUT	~50 MB (React build + images)	$0.00
Amazon CloudFront	1 TB data transfer out	~5 GB (academic testing traffic)	$0.00
MongoDB Atlas (M0)	512 MB storage	~20 MB (users + posts data)	$0.00

**Note: The Application Load Balancer is not covered under the standard Free Tier. However, new AWS accounts receive 750 hours of ALB usage in the first 12 months. An alternative is to configure HTTPS directly on the EC2 instance using a free Let's Encrypt certificate, reducing the cost to $0.00.*

Table 3.2: Estimated Monthly Cost Summary

Scenario	Estimated Monthly Cost
Development Phase (Free Tier, no ALB)	$0.00
Development Phase (Free Tier, with ALB within free hours)	$0.00
Post-Free Tier (minimal usage, EC2 + S3 + CloudFront)	~$8.50 – $12.00
Post-Free Tier (with ALB)	~$25.00 – $30.00

Conclusion: The financial implication of deploying EchoMateLite using this architecture is practically zero for the duration of the capstone project. The architecture leverages the AWS Free Tier strategically, selecting t2.micro for compute, M0 for database, and S3 for storage – all of which have generous free allocations. This demonstrates that production-grade cloud architectures can be designed and operated at minimal cost, making cloud deployment accessible for academic projects and startup prototypes alike.

REFERENCES

[1] Amazon Web Services. (2026). Amazon EC2 – Elastic Compute Cloud. https://aws.amazon.com/ec2/

[2] Amazon Web Services. (2026). Amazon S3 – Simple Storage Service. https://aws.amazon.com/s3/

[3] Amazon Web Services. (2026). Amazon CloudFront. https://aws.amazon.com/cloudfront/

[4] Amazon Web Services. (2026). AWS Free Tier. https://aws.amazon.com/free/

[5] Amazon Web Services. (2026). AWS Identity and Access Management (IAM). https://aws.amazon.com/iam/

[6] Amazon Web Services. (2026). Elastic Load Balancing. https://aws.amazon.com/elasticloadbalancing/

[7] Express.js. (2026). Express – Node.js Web Application Framework. https://expressjs.com/

[8] Meta Platforms, Inc. (2026). React – A JavaScript Library for Building User Interfaces. https://react.dev/

[9] MongoDB, Inc. (2026). MongoDB Atlas – Database as a Service. https://www.mongodb.com/atlas

[10] MongoDB, Inc. (2026). MERN Stack Explained. https://www.mongodb.com/mern-stack

[11] Node.js Foundation. (2026). About Node.js. https://nodejs.org/en/about/

[12] Tilkov, S., & Vinoski, S. (2010). Node.js: Using JavaScript to Build High-Performance Network Programs. IEEE Internet Computing, 14(6), 80–83.

ANNEXURE

Annexure A: Screenshots of Project Implementation

The following screenshots provide concrete evidence of the project implementation progress at this interim stage. These screenshots are captured from the active project workspace.

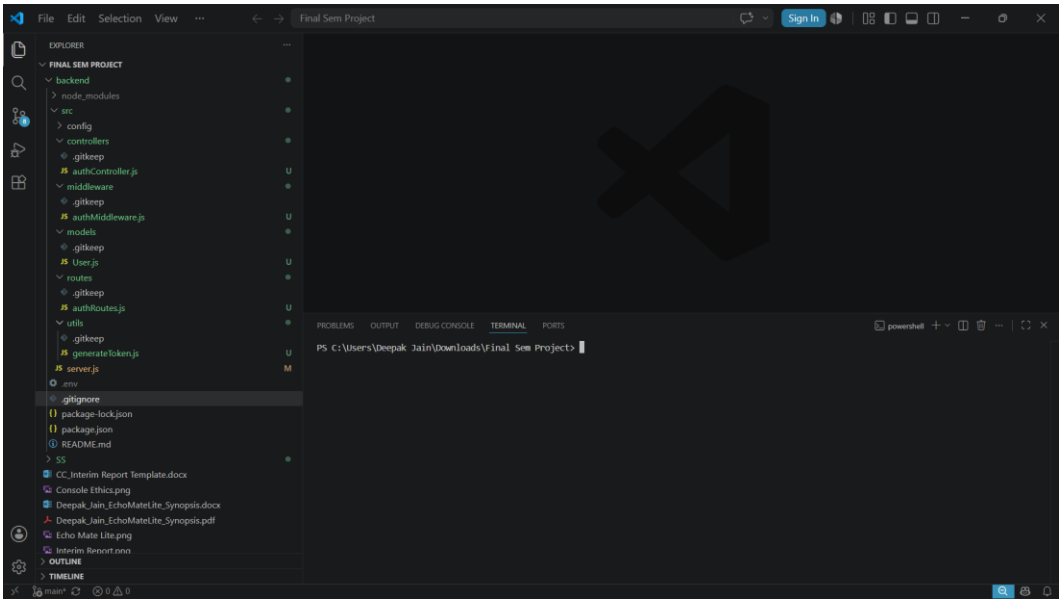

Figure A.1: VS Code – MVC Backend folder structure showing config, controllers, middleware, models, routes, and utils directories.

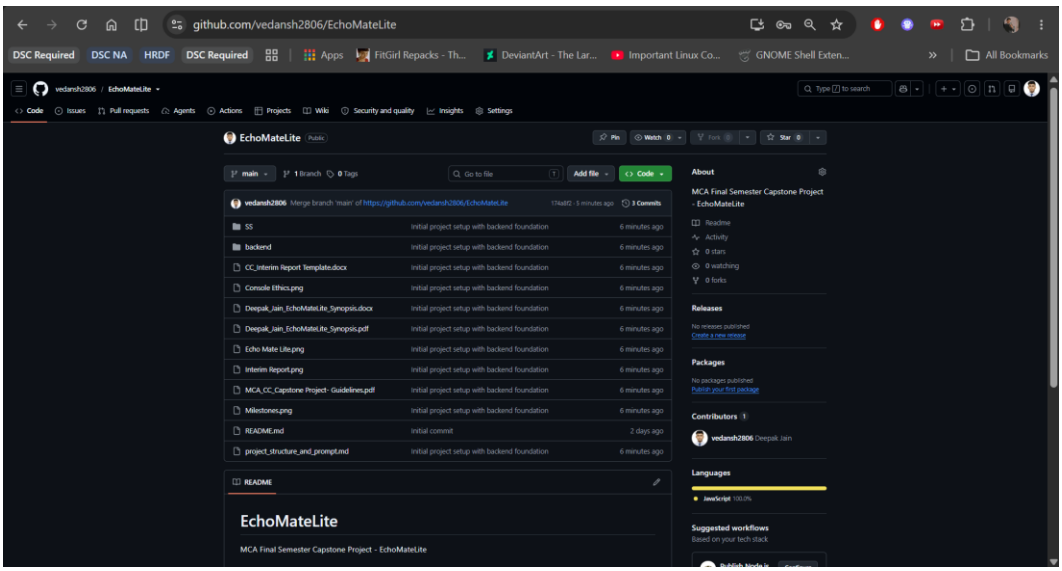

Figure A.2: GitHub repository (vedansh2806/EchoMateLite) showing 3 commits and the complete project file listing.

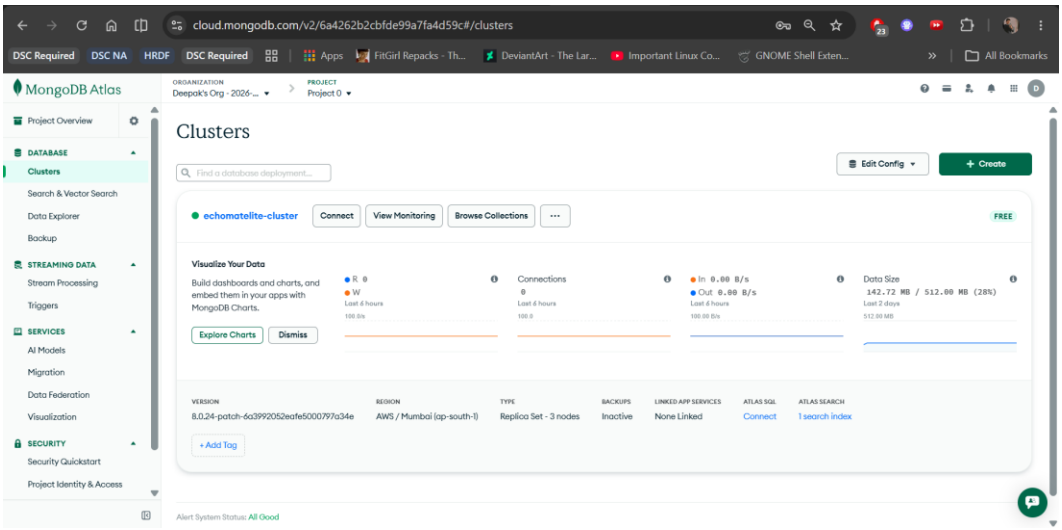

Figure A.3: MongoDB Atlas dashboard showing the "echomatelite-cluster" active on AWS Mumbai (ap-south-1) with M0 Free Tier, 142.72 MB / 512 MB usage, Replica Set with 3 nodes.

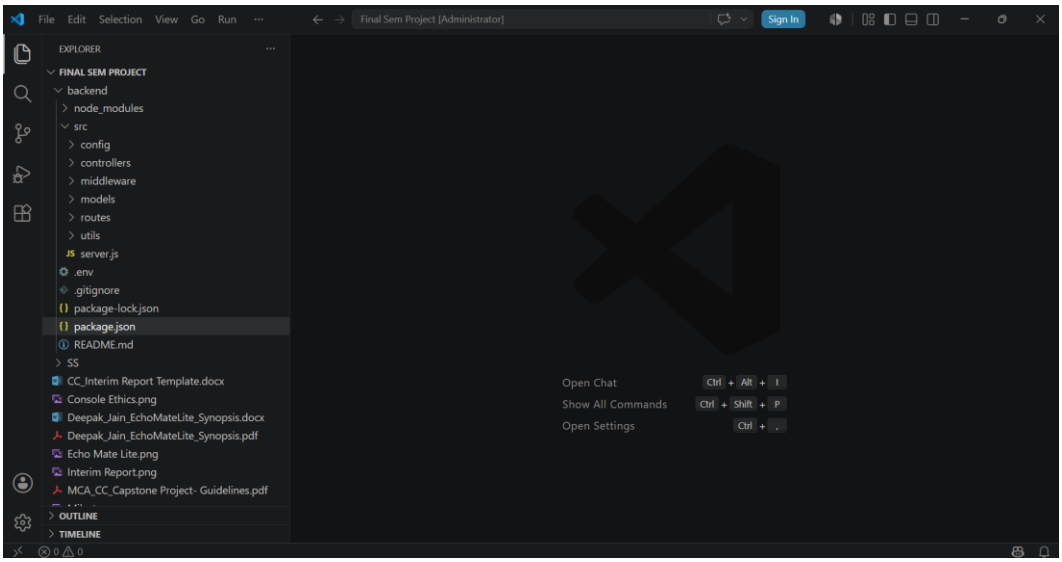

Figure A.4: VS Code explorer showing the complete User Authentication module: authController.js, authMiddleware.js, User.js model, authRoutes.js, and generateToken.js.

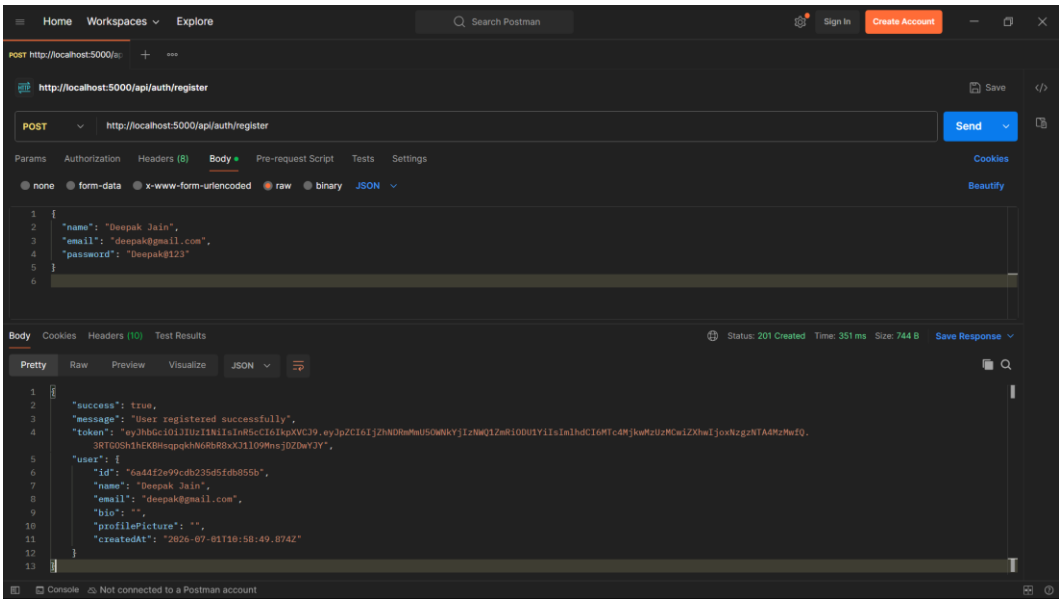

Figure A.5: Postman API test for POST /api/auth/register showing successful 201 Created response with JWT token, user object, and timestamp.

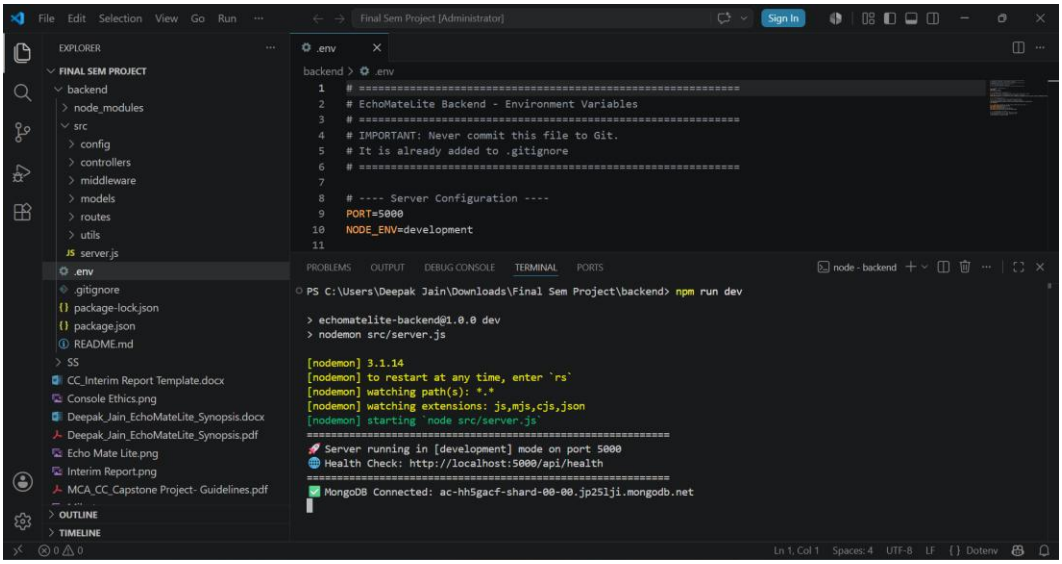

Figure A.6: VS Code terminal showing successful "npm run dev" execution with server startup banner and "MongoDB Connected" confirmation message.